2024

AutoConnect

MDP COURSEWORK 2

MOHAMMED SAMIR AHMED DASOUQI

Table of Contents

Introduction and Objective

The **AutoConnect App** is a mobile application that offers users tow services with a workshop catalogue, connecting users with workshop's seamlessly while providing tow services. The idea is that your car doesn't have to be undrivable to tow it, maybe you want the convivence of someone taking your car to the workshop and back when it is done.

The primary goal is to provide a seamless platform for users to locate nearby workshops and book tow truck services.

Target Audience

- Vehicle owners who require quick access to convenient repair services or towing solutions.
- Tow truck operators and workshops seeking an efficient system to connect with customers.

Technical Design and Implementation Details

Due to the feature rich nature of this application, it replies heavily on external APIs and services. This approach ensures that the used methods are tested, which in return means more reliable and more accurate. Below are the external services used:

- **Firebase Authenticaion:** for user authentication and profiles
- **Firestore database:** used to store user information, workshops and their details, tow requests, and completed tow orders. It is also used for live tracking of driver location under tow requests.
- **Google Maps SDK:** used to display maps in TowScreen and DriverScreen
- **Google Places API:** used to fetch specific workshop details like ratings and working hours. It is also used for autocomplete when the user is entering a location for tow requests.

- **Google Directions API:** used to handle all route related operations like calculating etas, distances, and the route itself (polyline).

The app is structured using a **modular and service-oriented architecture**, focusing on logical separation of concerns for ease of maintenance and scalability. The key layers and their responsibilities are outlined below:

1. Data Layer

   The data layer is divided into 3 categories:

   - Contains **models** (DirectionsResponse, Workshop, Order) that define the core data entities used across the app.
   - The **remote** subpackage manages interactions with external APIs using Retrofit, while **repository** modules centralize data access logic, ensuring a single source of truth.
   - The **repository** contains WorkshopRepository which is responsible to fetch all the workshops saved in the database.

2. Service Layer

   The Services Layer is responsible for managing core functionalities and abstracting complex operations from the UI. This layer is organized into modules to ensure modularity, scalability, and separation of concerns. Below is a list of the services:

   - **AuthenticationService:** houses all the operations related to user accounts like sign up, log in, getting user instance, etc.
   - **DirectionsService:** defines methods for interacting with a directions API using Retrofit.
   - **LocationServices.kt**: manages geolocation-based tasks and handles required permissions.
   - **LocationUpdateService.kt:** provides background location updates while tow order is active, required for the driver.

- **NotificationService.kt:** manages notifications along with required permissions.
- **PlacesSdkService.kt:** Initializes and communicates with Google's Places SDK.
- **TowService.kt**: Handles all operations related to tow requests for both the user and driver, including real-time updates and data persistence in Firebase Firestore.

3. UI Layer

The UI layer is divided into 3 distinct packages, representing functional sublayers that organize the user interface components. Here's how these packages are structured:

Components

The **components** sublayer contains reusable UI components that could be used across multiple screens. This sublayer contains:

- **CustomTopAppBar.kt:** a customer top bar that is always on the screen, it houses 3 elements: profile drawer button, current page title, and notifications button.
- **MapViewComponent.kt:** a map UI component to integrate Google Maps. It is used in TowScreen and DriverScreen.
- **ProfileDrawer.kt:** the profile drawer itself, extending on CustomTopAppBar, it can be triggered in any screen.
- **TechnicianGrid.kt:** used in WorkshopDetailsScreen, it shows the technicians that are available in a specific workshop in grid style.
- **WorkshopDetailsBottomBar.kt:** this is a bar similar to CustomTopAppBar, it is used in WorkshopDetailsScreen to house 3 button on the bottom of the screen.

Screens

The **screens** sublayer houses all the screen layouts of the app. There of a total of 8 screens in the first version of this application. These screens are:

- **DriverScreen.kt:** displays a list of tow requests for a driver. It uses various state variables to manage the loading state and the list of tow requests.

- **HomeScreen.kt:** display the home screen of the application. It interacts with Firebase Authentication and Firestore to fetch user details and filter appropriate buttons for specific user types.
- **LoginScreen.kt:** responsible for rendering the login screen of the application. It creates a user interface that includes text fields for email and password input, a "Forgot password?" link, and a login button.
- **OrderScreen.kt:** displays a list of completed tow orders for the authenticated user. It uses Firebase Firestore to fetch the orders and Firebase Authentication to get the current user's details.
- **SignUpScreen.kt:** creates a user interface for a sign-up. Very similar to the login screen design wise.
- **TowScreen.kt:** handles the user interface for requesting a tow service. It manages the state and interactions related to selecting locations, displaying routes, and handling tow requests in correlation with TowService.
- **WorkshopBrowse.kt:** designed to display a scrollable list of workshops fetched from the database.
- **WorkshopDetailsScreen.kt:** designed to display detailed information about a workshop. It creates a UI layout that includes an image, title, review, description, and a list of technicians.

Theme

The **theme** sublayer is the last sublayer of the UI layer. It manages the UI styling and theming of the app. This sublayer is generated by android studio by default. Currently it is not being fully utilized to its potential because the application has only 1 theme and a simple style. There are 3 files in this sublayer:

- **Color.kt:** Defines the app's color palette.
- **Theme.kt:** Centralizes the Material Theme setup for consistent styling.
- **Type.kt:** Defines typography styles like font sizes and weights.

4. Utility Layer

The Utility layer contains helper classes and functions that provide support functionalities for various app operations. These utilities are divided into distinct files, each serving specific purposes and facilitating modularity and reusability across the application.

- **NavigationUtils.kt**: helps in navigating to a specified location using either Google Maps or Waze.
- **OpeningHoursFormatter.kt:** formats opening hours from a string representation that is received from Places SDK into a more readable format.
- **RouteUtils.kt:** provides helper functions for route calculations, such as polyline decoding and distance formatting.

Specific Design Patterns and Advantages

1. Modular Architecture

The app follows a modular structure where distinct layers — UI, Services, and Utilities — are clearly separated. This separation enhances **maintainability**, as changes in one layer do not affect others, and enables **scalability**, making it easier to add new features.

2. Component-Based UI Design with Jetpack Compose

Instead of the traditional XML-based layouts, the app uses Jetpack Compose, a modern toolkit for building native UI in Android. Key advantages include:

- **Declarative Syntax:** Simplifies UI development by allowing developers to describe "what" the UI should look like, rather than "how" to construct it.
- **Reusability:** UI components (like CustomTopAppBar or TechnicianGrid) are modular and reusable across screens.

- **State Management:** Integration with Compose's state system ensures a responsive UI that updates automatically when state changes.

3. Observer Pattern for Real-Time Updates

Firebase Firestore listeners implement the **Observer pattern**, allowing the app to react to real-time changes, such as tow request status updates or location changes.

Analysis of Challenges and Solutions

This project presented several challenges, mainly due to the size and complexity of the features and the technical demands of integrating external services. Below is an analysis of the key challenges faced during the development process and the solutions employed to address them.

Underestimating the Scope of the Project

The scale of the application, with its different types of features like real-time location tracking, Google Maps integration, background services, and user notifications, was underestimated. This led to a significant workload, especially when combined with time constraints and other academic commitments.

To address this, I adopted a modular approach, breaking the application into smaller, manageable components. By prioritizing features based on their criticality and potential user impact, I was able to ensure incremental progress. Time management strategies, such as setting micro-deadlines for individual modules, were essential in keeping the project on track.

Integrating Google Maps with Location and Live Tracking

Implementing Google Maps along with location services and live tracking was the most technically demanding aspect. Understanding how to connect Google Maps, location updates, and Firebase Firestore for real-time tracking was overwhelming.

To tackle this, I broke the problem into smaller, logical steps:

1. First, I implemented basic Google Maps functionality, ensuring proper initialization and display.
2. Then I added location services to retrieve and store the driver's location in the database.
3. Finally, I connected the live tracking to read the driver's location in the database every 30 seconds, synchronizing location updates between users and drivers. By taking it step by step, brainstorming potential connections between the components, and relying on documentation and tutorials, I managed to achieve seamless integration.

Implementing Background Services

Implementing background services for notifications and location updates was particularly difficult because these features were considered late in the development process. Without proper initial planning, integrating these services at the end introduced technical complexities and potential stability issues.

I had to revise core parts of the application's code to accommodate background services, rearranging and connecting the code.

Lessons Learned

More than half way through the development progress, nearing the deadline, I realized that it is impossible to implement all of what I had in mind. This is not due to my inability to do so, but due to the size and number of features I had in mind. I kept getting ideas and learned that if I wanted to do everything, the application development will never end. That is why developers usually release an application and keep adding feature during its lifespan. Possible implementations and improvements are endless.

The most important thing I learned is the importance of planning for such features from the beginning of the development cycle to avoid rework, maintain code consistency, increase efficiency, and decrease cost.

Testing and Validation

The application has been manually tested to ensure functionality and reliability. Although automated tests were not implemented during the development process, manual testing was conducted across various features to identify and fix issues.

Manual Testing Approach

- **Functional Testing:** Each feature, such as Google Maps integration, real-time tracking, and authentication, was tested to ensure it worked as expected.
- **UI Testing:** User interfaces were manually reviewed on different devices to verify responsiveness and usability.
- **Integration Testing:** The interaction between various components, such as Google Maps, Firebase, and background services, was manually tested to ensure seamless functionality.

Observations and Results

- **Reliability:** Manual testing confirmed the application's core features function as intended under normal use cases.
- **Usability:** Feedback from initial testing sessions indicated that the app's interface is user-friendly and intuitive.

Future Plans

Implementing automated tests for unit, integration, and UI testing is a priority for future development to ensure more robust validation and scalability. This hasn’t been implemented purely due to time constraints, features have been prioritized.

Appendix

GitHub Repository Link

“kczenb” has been invited to the repo as it is private.

Project File Structure

```
autoconnect/
├── AutoConnectApp.kt
├── MainActivity.kt
├── data/
├──   ├── models/
├──   ├──   ├── DirectionsResponse.kt
├──   ├──   ├── Order.kt
├──   ├──   ├── Workshop.kt
├──   ├── remote/
├──   ├──   ├── RetrofitInstance.kt
├──   ├── repository/
├──   ├──   ├── WorkshopRepository.kt
├── services/
├──   ├── AuthenticationService.kt
├──   ├── DirectionsService.kt
├──   ├── LocationServices.kt
├──   ├── LocationUpdateService.kt
├──   ├── NotificationService.kt
├──   ├── PlacesSdkService.kt
├──   ├── TowService.kt
```

```
├── ui/
├──   ├── components/
├──   ├──   ├── CustomTopAppBar.kt
├──   ├──   ├── MapViewComponent.kt
├──   ├──   ├── ProfileDrawer.kt
├──   ├──   ├── TechnicianGrid.kt
├──   ├──   ├── WorkshopDetailsBottomBar.kt
├──   ├── screens/
├──   ├──   ├── DriverScreen.kt
├──   ├──   ├── HomeScreen.kt
├──   ├──   ├── LoginScreen.kt
├──   ├──   ├── OrderScreen.kt
├──   ├──   ├── SignUpScreen.kt
├──   ├──   ├── TowScreen.kt
├──   ├──   ├── WorkshopBrowse.kt
├──   ├──   ├── WorkshopDetailsScreen.kt
├──   ├── theme/
├──   ├──   ├── Color.kt
├──   ├──   ├── Theme.kt
├──   ├──   ├── Type.kt
├── utils/
```

├── ├── NavigationUtils.kt

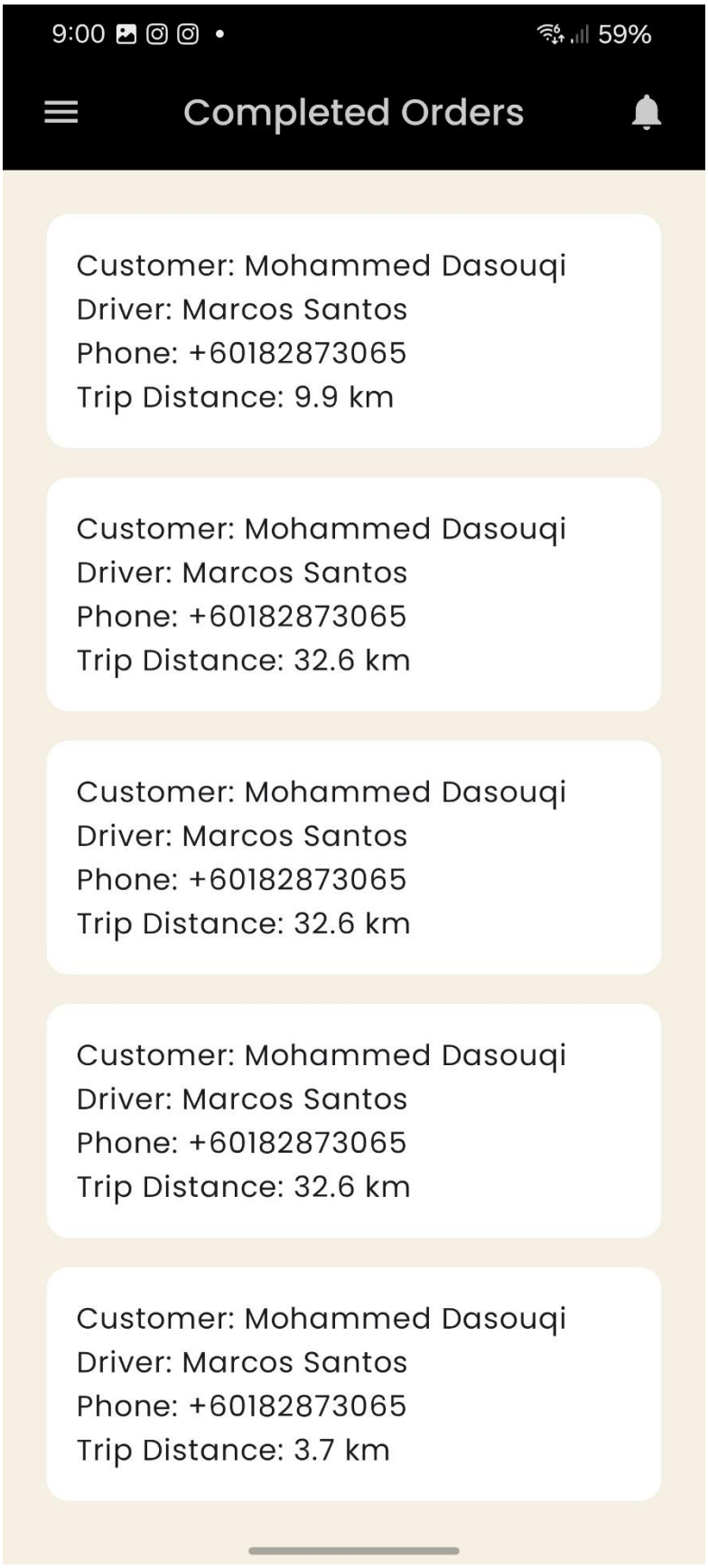
├── ├── OpeningHoursFormatter.kt

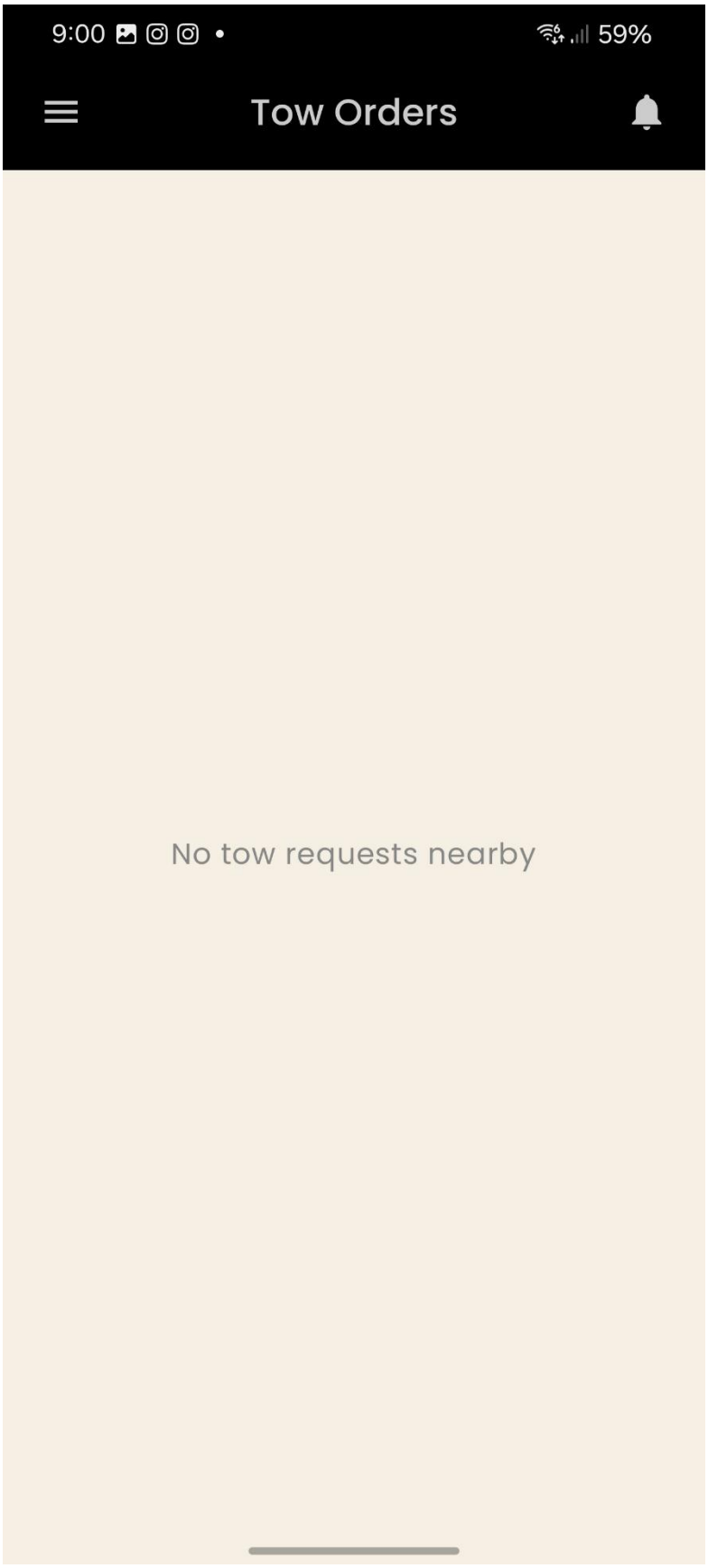
├── ├── RouteUtils.kt

├── viewmodel/

├── ├── WorkshopViewModel.kt

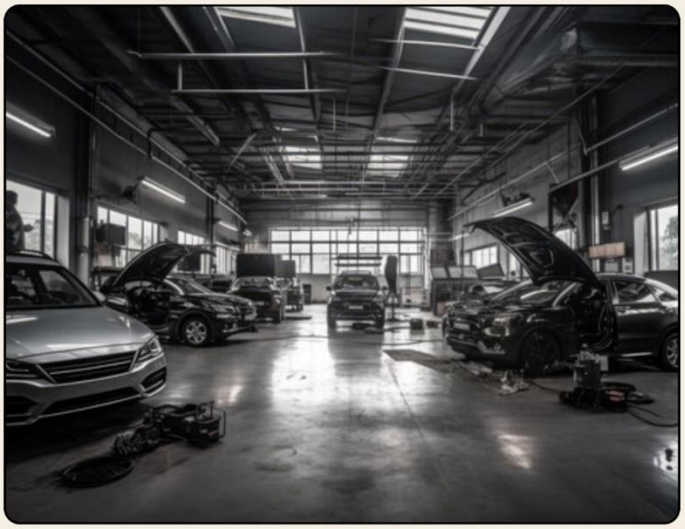
Screenshots


9:00
59%
Completed Orders
Customer: Mohammed Dasouqi
Driver: Marcos Santos
Phone: +60182873065
Trip Distance: 9.9 km
Customer: Mohammed Dasouqi
Driver: Marcos Santos
Phone: +60182873065
Trip Distance: 32.6 km
Customer: Mohammed Dasouqi
Driver: Marcos Santos
Phone: +60182873065
Trip Distance: 32.6 km
Customer: Mohammed Dasouqi
Driver: Marcos Santos
Phone: +60182873065
Trip Distance: 32.6 km
Customer: Mohammed Dasouqi
Driver: Marcos Santos
Phone: +60182873065
Trip Distance: 3.7 km


9:00
59%
Tow Orders
No tow requests nearby

8:59 • 59%

Workshop Details



Speed Auto

★★★★★ 4.9

Description

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut

Technicians

- ✓ Electrician
- ✓ Mechanic
- ✓ AC Specialist

Navigate | Request Quote

8:59 • 59%

Browse Workshops

Speed Auto

Klang ★ 4.9

Closed

9:00 AM - 5:30 PM

Navigate

8:59 • 59%

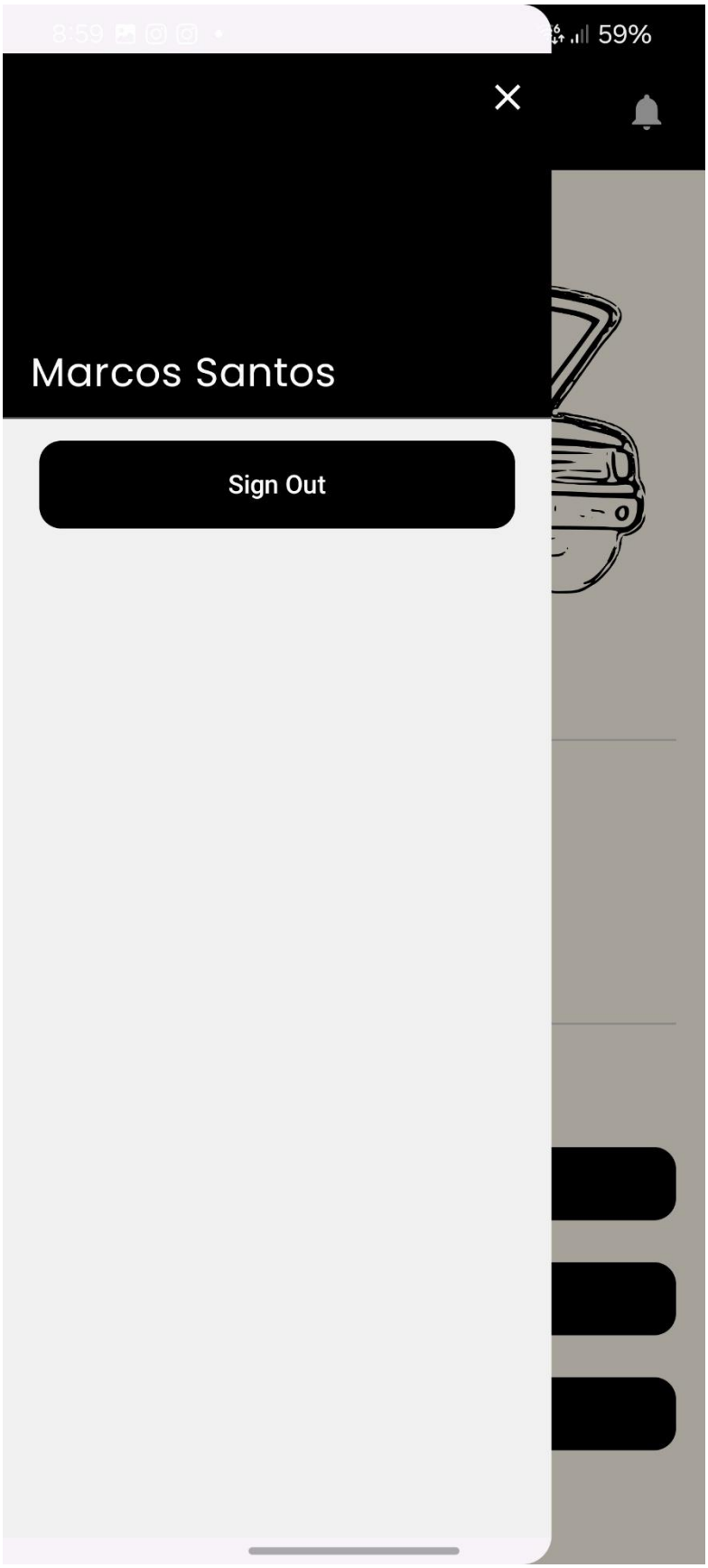
Home

Welcome Marcos

Browse Workshops

Driver Screen

Completed Orders


8:59
59%
Marcos Santos
Sign Out

8:58 • 59%

Home

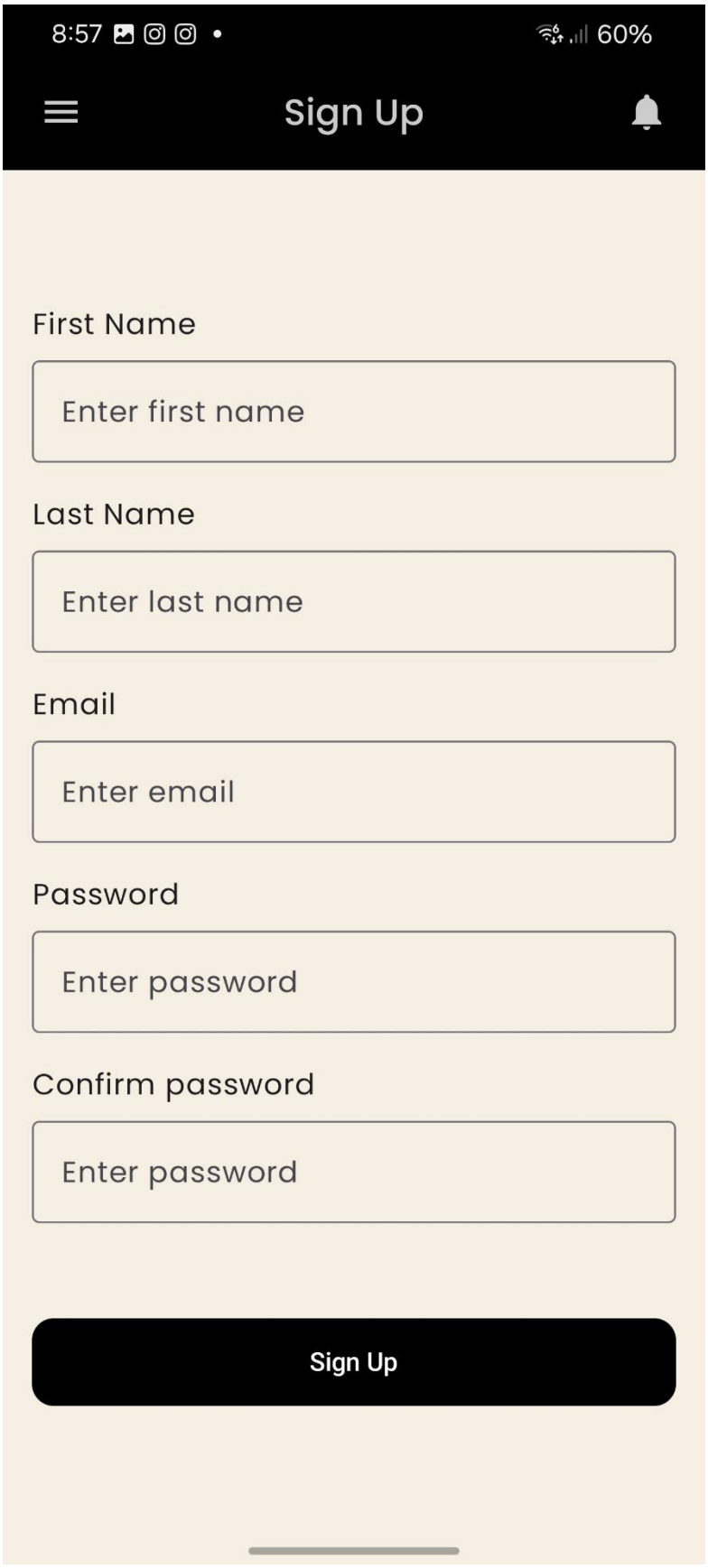
Welcome Admin

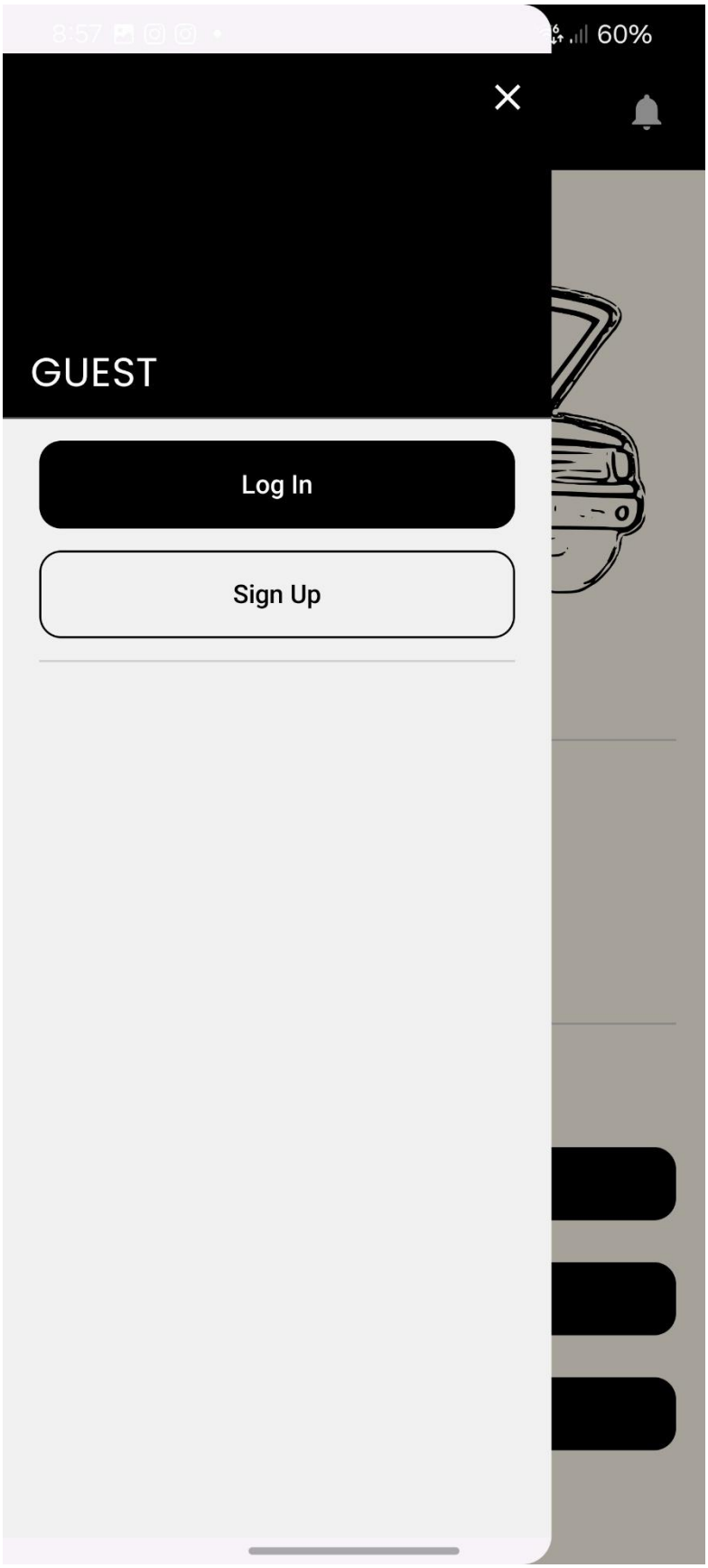
Browse Workshops

Request Tow

Driver Screen

Completed Orders


8:57
60%
Sign Up
First Name
Enter first name
Last Name
Enter last name
Email
Enter email
Password
Enter password
Confirm password
Enter password
Sign Up


8:57
60%
GUEST
Log In
Sign Up

8:56 • 60%

Home

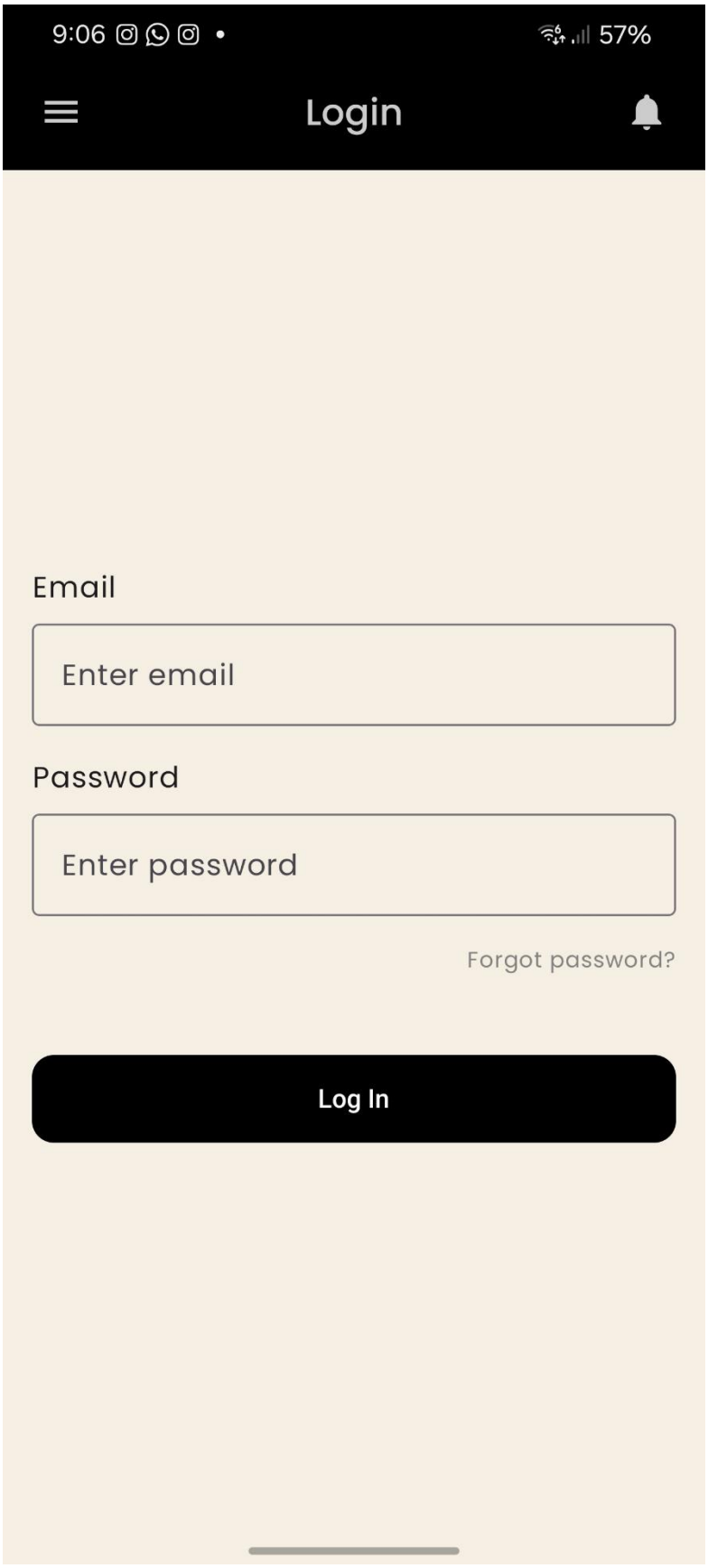
Welcome Guest

Browse Workshops

Request Tow

Completed Orders

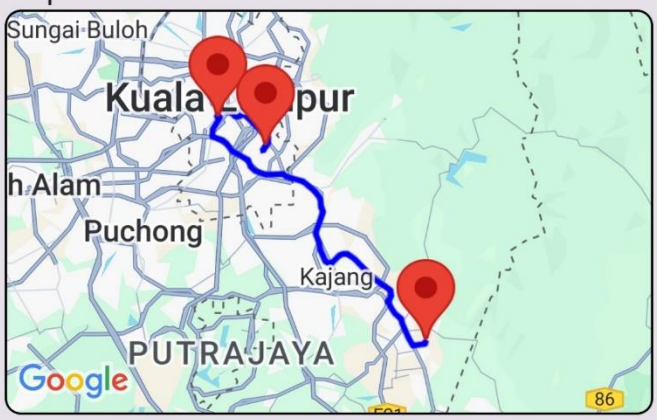

Auto Connect


9:06
57%
Login
Email
Enter email
Password
Enter password
Forgot password?
Log In

9:04 • 57%

Tow Orders

Name: Mohammed Dasouqi
Distance to Pick Up: 38.5 km
Time to Pick Up: 47 mins
Trip Distance: 9.9 km
Trip Estimated Time: 13 mins



Total Route Distance: 48 km
Total Estimated Time: 59 mins

Cancel

Navigate to Pick Up

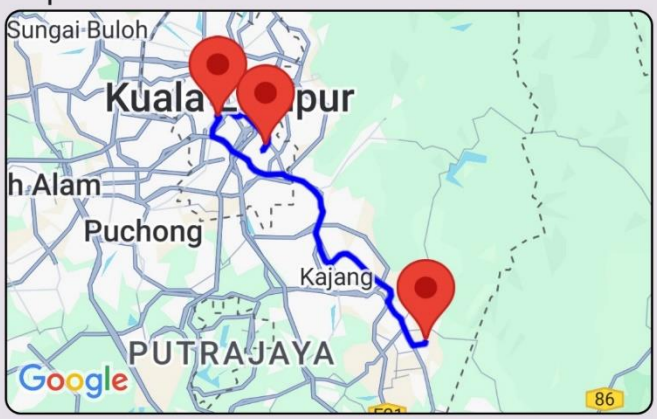
Navigate to Drop Off

Complete Job

9:04 • 58%

Tow Orders

Name: Mohammed Dasouqi
Distance to Pick Up: 38.5 km
Time to Pick Up: 47 mins
Trip Distance: 9.9 km
Trip Estimated Time: 13 mins



Total Route Distance: 48 km
Total Estimated Time: 59 mins

Cancel

Navigate to Pick Up

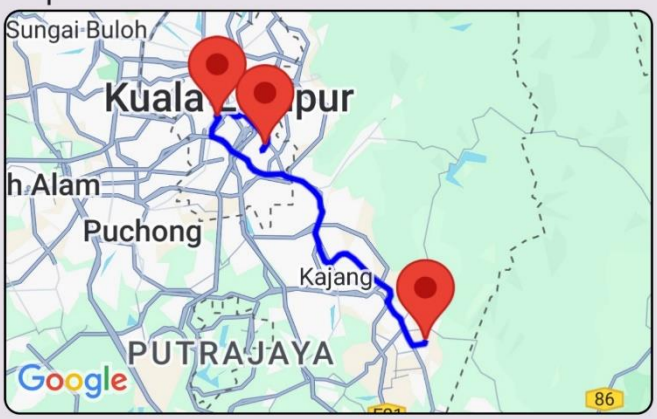
Navigate to Drop Off

Picked Up

9:04 • 58%

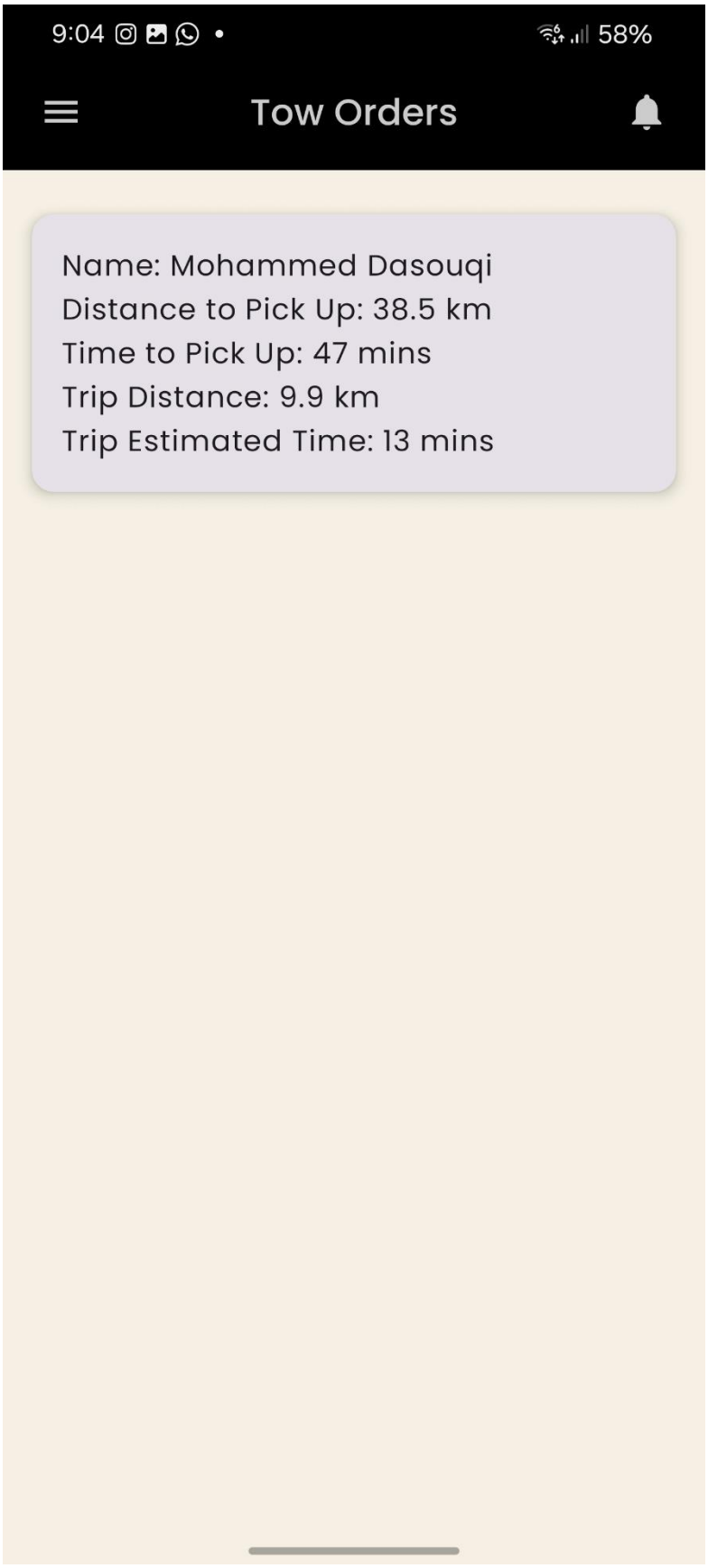
Tow Orders

Name: Mohammed Dasouqi
Distance to Pick Up: 38.5 km
Time to Pick Up: 47 mins
Trip Distance: 9.9 km
Trip Estimated Time: 13 mins



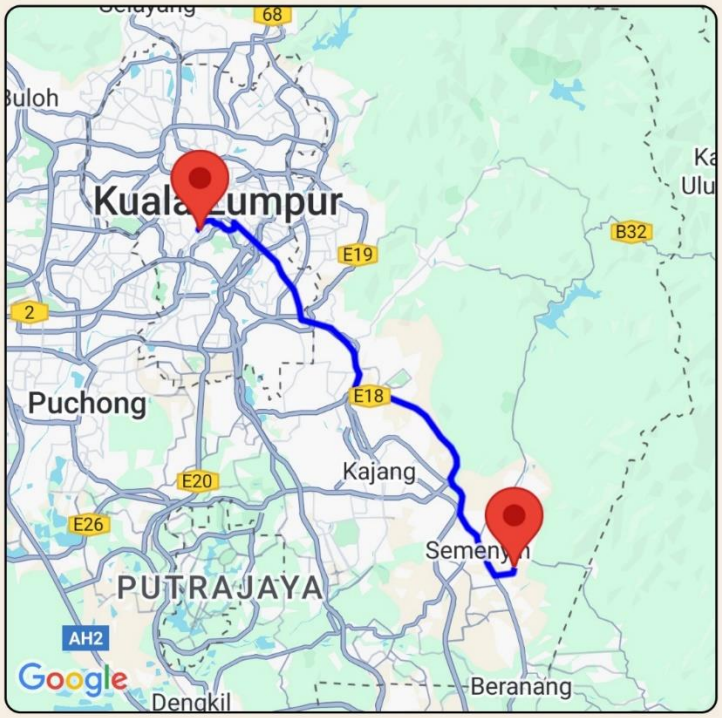
Total Route Distance: 48 km
Total Estimated Time: 59 mins

Accept


9:04
58%
Tow Orders
Name: Mohammed Dasouqi
Distance to Pick Up: 38.5 km
Time to Pick Up: 47 mins
Trip Distance: 9.9 km
Trip Estimated Time: 13 mins

9:03 • 58%

Tow Service



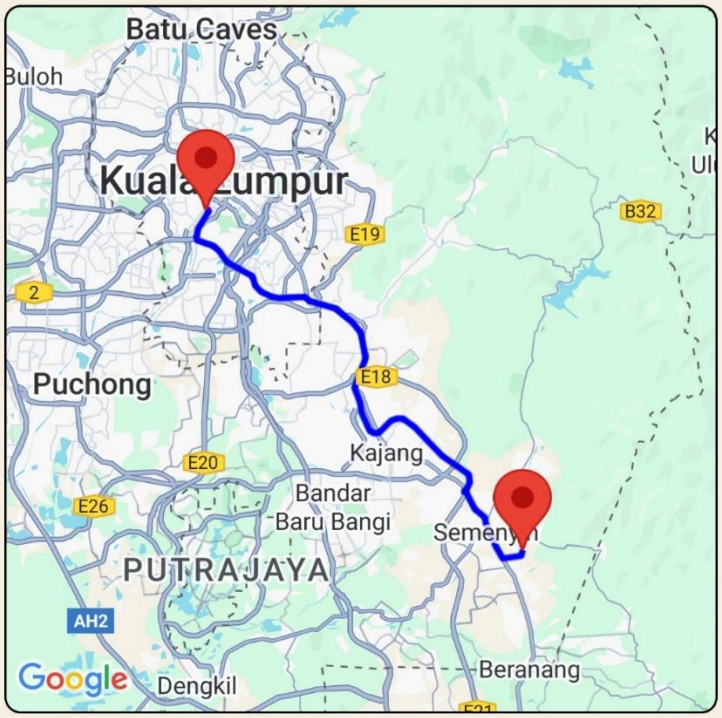
Estimated Trip Time: 44 mins
Trip Distance: 37.8 km

Marcos Santos

Cancel

9:00 • 58%

Tow Service



Use Current Location

Federal Territory of Kuala Lumpur

Estimated Trip Time: 47 mins
Trip Distance: 38.5 km

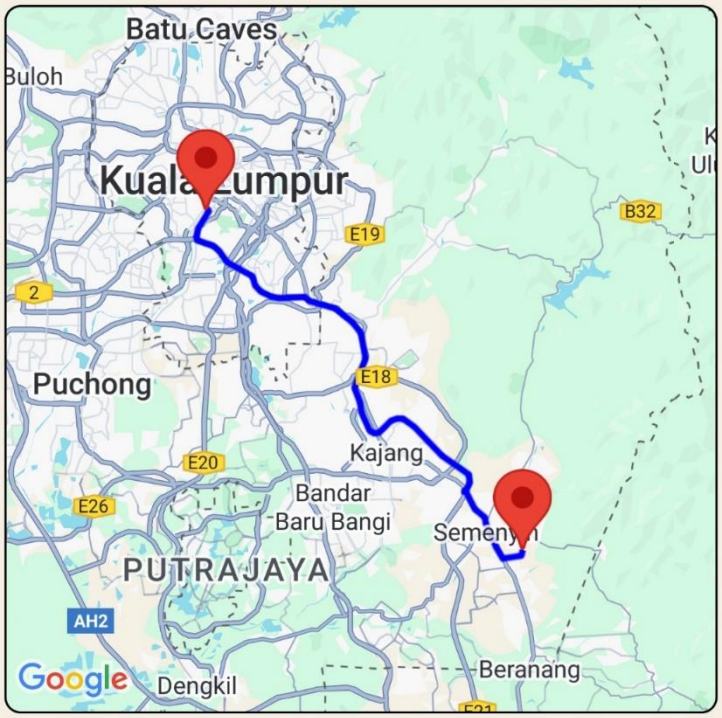
Looking for a Tow Truck

Cancel

Please do NOT close this page or your request will be canceled.

9:00 • 58%

Tow Service



Current Location

Use Current Location

Federal Territory of Kuala Lumpur

Estimated Trip Time: 47 mins
Trip Distance: 38.5 km

Send Route to Nearby Drivers

9:00 • 58%

Tow Service

Pick up

Use Current Location

Drop off